

Myapp Architecture

Document Version: V 0.1

Primary Author: Vishal Keshava Murthy

Document source: [\[\[LINK\]\]](#)

Version History

Version	Revision Date	Comment	Revised By
0.1	28-06-2026	Initial Version	VKM

Index

Content	Page No.
Index.....	0
Context.....	1
References.....	1
Architecture overview.....	1
Architecture Detailed.....	1
Server.py.....	1
Server_handles.py.....	2
UDP.....	2
TCP.....	2
TLS.....	2
Server_config.py.....	2
Server_logging.py.....	2
Server_authenticator.py.....	2
Server_payload_validator.py.....	3
Server_rate_limiter.py.....	3
Appendix.....	3

Context

This document captures the architecture of server implementation developed as a part of technical evaluation. Only the server is documented, the project ReadMe file covers details about project usage and other details

References

1. Assignment_edge.pdf
2. Server code base

Architecture overview

The Server module is built up as a collection of dedicated modules with defined functionality. The sections below will go over each file and explain key design decisions. At a glance the files and their responsibilities are

3. server.py : Module entry point, runs EchoServer class which processes messages
4. server_handlers.py : tcp, udp and tls connection management is handled here
5. server_config.py : Manages configurable parameters for the servers through .env
6. server_logging : All logging capabilities for the server here
7. server_authenticator : Authentication of the client happens here, currently as JWT
8. server_payload_validator : Payload validation functions for size and format here
9. Server_rate_limiter : client rate limits in as set though .env implemented here in the form of a Fixed window counter

Architecture Detailed

Server.py

This is the function entry point, which initializes and runs the **EchoServer** class. Services such as configuration, logging, authentication, payload validation and rate limiting are delegated to the respective sub-modules.

At `__init__` the respective handlers for TCP, UDP and TLS are set up and invoked.

At the modules core is the `run` method which waits on events from the sockets, to invoke the registered callbacks for the respective events in `server_handles.py`

The handlers send the messages back to [process_message](#) which performs ratelimiting, payload validation checks before echoing the response back

Server_handles.py

Implements an abstract baseclass [BaseProtocolHandler](#) , inherited by concrete classes [TCPHandler](#), [UDPHandler](#) and [TLSHandler](#).

On Setup, respective sockets are opened and set to listen waiting for clients. The EVENT_READ is registered and associated with the protocol specific connection handler. In

UDP

- ON_MESSAGE -> _handle_read -> server.process_message

TCP

- ON_CONNECTION_REQUEST | _handle_accept
- ON_DATA | _handle_read -> server.process_message

TLS

- ON_CONNECTION_REQUEST | _handle_accept (Wrap TCP with SSL context) -> _handle_handshake
- ON_HANDSHAKE_SUCCESS | _handle_auth
- ON_AUTH_SUCCESS | _handle_client_data -> server.process_message

Server_config.py

server_config imports the .env file through the [dotenv](#) library. Defaults are also defined in this file in case the env parameter is missing. Host, ports, certificate paths, log file paths, payload size, and rate limits are managed here

Server_logging.py

All logging in the application is centralized through a module built on top of python inbuilt [logging](#). [Configure_logging](#) method invoked in each module takes parameters for logger name, log level, and an optional logging filter to provide granular logging at a module level.

[AccessControlFilter](#) provides filtering capabilities by restricting records based on client or role.

Server_authenticator.py

Authenticator handles client-authentication and houses a virtual base class [Authenticator](#) and concrete implementation [JWTAuthenticator](#).

The `JWTAuthenticator` is built atop the `jwt` module. It uses the secret and algorithm as defined in the `.env` to decode incoming packets and authenticate the same.

If more Authentication strategies are developed in the future a wrapper service can be developed that delegates the authentication to the selected underlying concrete authentication strategy.

Server_payload_validator.py

`validate_payload` method to check for size and decode errors.

Server_rate_limiter.py

Rate limiter enforces rate-limits and houses a virtual base class `RateLimitStrategy` and concrete implementation `FixedWindowCounter`.

The `FixedWindowCounter` defines `is_allowed` method which maintains a dictionary, associating `client_id` with a tuple of message count and time-window. If the message exceeds the limit under the time-window for any client, it is rejected.

If more `rate_limit` strategies are developed in the future a wrapper service can be developed that delegates the operation to the selected underlying concrete rate-limiting strategy.

Appendix